

Программирование и алгоритмизация

(МИСиС, осень-зима 2025)

Полевой Дмитрий Валерьевич
к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

ваши вопросы

Встраивание функций (`inline`)

- рекомендация компилятору встроить инструкции «по месту» без «вызова»
- управляется ключами компиляции
- уменьшение времени исполнения
- увеличение объема исполняемого кода
- усложнение отладки

Встраиваемая функция

- определение д.б. доступно в любой точке вызова и м.б. определена в заголовке
- имеет особенности связывания

пример:

```
// in utility.h
```

```
inline int sum(const int a, const int b) {  
    return a + b;  
}
```

Встраиваемые методы (внутри типа)

- определены в определении типа

пример:

```
class Array {  
    ...  
    int size() const { return size_; }  
    ...  
};
```

Встраиваемые методы (вне типа)

```
// in array.h  
class Array {  
    int size() const;  
};  
  
inline int Array::size() const {  
    return size_;  
}
```

ваши вопросы

Висячая ссылка

(dangling reference)

- невалидная ссылка
- м.б. получена возвратом ссылки на объект, который разрушается раньше ссылки
 - локальный (стековый) из функции
 - на динамический и разрушенный «руками»
- избегать создания таких ситуаций

Висячая ссылка (пример)

```
int& GetValue() {  
    int val = 0;  
    return val;  
}
```

```
auto v = GetValue();  
v += 1;
```

Глобальная переменная

использование минимизировать

- определяется вне любого блока
- создается в начале работы программы
- уничтожается после завершения работы
- делать `static` и доступ через функцию
- допустимо `static const` для «параметров»

extern (ключевое слово)

- тип хранения (storage class)
- объявление внешней функции или переменной

пример:

```
// в a.cpp
```

```
int gObjectsCounter(0);
```

```
// в b.cpp
```

```
extern int gObjectsCounter;
```

Использование `extern`

- только при большой необходимости
 - поддержка старой кодовой базы
 - стыковка со сторонними библиотеками
- чревато
 - тонкие ошибки проверки типов
 - неконтролируемый доступ к общим данным

`static` (ключевое слово)

- тип хранения (storage class)
- применим к переменным и функциям
- видимость – единица трансляции
- время жизни – время жизни программы

Статические переменные

- память выделяется в специальной области
- локальная переменная
 - «конструируется» при первом обращении
 - сохраняет значение между вызовами

Статическая переменная (пример)

```
// в point.cpp
static int gModuleCallsCount = 0;

int inside(const Pt& p, const Rect& rc) {
    static int nFuncCallsCount = 0;
    nFuncCallsCount += 1;
    gModuleCallsCount += 1; // плохо
    ...
}
```

Статический член типа

- является частью типа, но не экземпляра
- доступ
 - через экземпляр
 - через имя типа
- д.б. где-то определен
(единственным образом)

Статический член класса (пример)

```
// в point.h  
class Point {  
    public:  
        static int GetRefPoint() ;  
    private:  
        static int m_xRefPoint;  
};
```

```
// в point.cpp  
int Point::m_xRefPoint = 0;
```

Доступ к статическому члену типа

- через экземпляр

```
Point point;  
point.GetRefPoint();
```

- через имя класса

```
Point::GetRefPoint();
```

Время жизни статических объектов

- порядок создания и уничтожения
 - до C++11 не был определен между разными единицами трансляции
 - может быть источником ошибок (сложно понять и отладить)

ваши вопросы

Указатель на функцию

```
ReT (*T) ();
```

```
T pF(funcName);
```

```
ReT result(pF());
```

например:

```
int (*pFunc)(float, int);
```

```
int* (ClassT::*pMemb)(double);
```

Какие опасности?

- слабый или отсутствующий контроль типов
- накладные на вызовы функций
- глобальные данные

Зачем такой «ужас»?

- реализация динамического поведения
 - функции обратного вызова (callback)
 - подгрузка исполняемого кода
(динамические библиотеки)
- повторное использование кода
- архитектурная гибкость

Чем заменять указатели на функцию?

- функторы
 - тип с перегруженным `operator()`
 - обычно, `struct`, т.к. должны не иметь состояния
- лямбды

Функтор (объект-функция)

- пользовательский тип
- перегружает оператор вызова функции
`operator()`
- используется для параметризации алгоритмов
- может встраиваться компилятором в тело алгоритма (в отличие от обычной функции)

Функтор

(пример)

```
struct StrLenCmp {  
    bool operator()(const string &lhs,  
        const string &rhs) const {  
        return lhs.length() < rhs.length();  
    }  
};
```

Стандартные функторы

- заголовок `<functional>`
- `equal_to`, `not_equal_to`, `less`, `greater`, И Т.Д.
- `plus`, `minus`, `multiply`, `divides`, И Т.Д.
- `logical_and`, `logical_not`, И Т.Д.
- И Т.П.

Обертка функции `std::function`

- заголовок `<functional>`
- полиморфное использование функции
 - указатель на функцию
 - лямбда
 - привязка выражения (`std::bind`)
 - функтор
- хранение, передача, вызов

Лямбда-выражение

- определяет (локальный) анонимный тип
- экземпляр можно хранить
- экземпляр можно копировать (передавать)
- экземпляр используется для вызова функции

Определение лямбда-выражения

`[] () {}`

`[]` - список захвата (м.б. пустым)

`()` - список аргументов (м.б. пустым)

`->` тип возвращаемого значения (необязательно)

`{}` - блок кода (м.б. пустым)

Тип возвращаемого значения для лямбды

- суффиксный стиль ($\rightarrow$)
- если опущен
 - для тела в виде `return` вычисляется по типу возвращаемого выражения
 - если `return` несколько – д.б. один тип
 - `void`, если опущено возвращаемое значение

Список захвата (capture list)

- список и способ захвата локальных переменных (из области определения), которые могут использоваться внутри лямбда-выражения (помимо аргументов)
- умолчательно переменные захватываются как константные значения

Варианты захвата

[] — ничего не захватывается

[&v] — захват по ссылке переменной v

[=v] — захват по значения переменной v

[&] — захват всех переменных по ссылке

[=] — захват всех переменных по значению

Захват нескольких переменных

- индивидуальные спецификаторы (значение/ссылка)
- через запятую перечисляем
- каждая переменная один раз

пример:

```
[width, height, &type]
```

Как разрешить изменять захваченное

- снять спецификатора `const` с захваченных по значению локальных переменных

```
[ ] ( ) mutable { }
```

ваши вопросы

Висячие ссылки при захвате

- захваченные по ссылке переменные должны существовать дольше самой лямбды
- минимизировать захват по ссылке для хранимых лмбд
- избегать создания таких ситуаций

Захват с инициализацией

- новая локальная переменная
- тип и значение вычисляется один раз в процессе компиляции

Захват с инициализацией (пример)

```
int x = 10;  
int y = 11;  
auto foo = [z = x+y] () {  
    std::cout << z << '\n'; };  
foo();
```

Умолчательные значения параметров

- аналогично обычным функциям

пример:

```
auto f = [] (int x = 10) {  
    cout << x << '\n'; };  
  
f();  
f(100);
```


Хранение лямбд

- помнить про уникальность типа
- использовать шаблон `function<T>`
- следить за копированием или перемещением лямбд

Хранение лямбд (пример)

```
vector<function<int()>> fs;  
fs.push_back([](int x) { return 2*x; });  
fs.emplace_back([](int) { return 1; });  
  
for (const auto& f: fs) {  
    cout << f(6) << ' ';  
}
```

Generic Lambda

- ключевое слово `auto` в списке параметров
- автогенерация кода для вычисленного типа

Generic Lambda

(пример)

```
auto PrintLn = [] (const auto& x) {  
    std::cout << x << '\n'; };  
  
PrintLn(10);  
PrintLn(10.1234);  
PrintLn("hello world");
```

Generic Lambda

(пример)

```
map<string, int> nrs{
    {"one", 1}, {"two", 2}, {"three", 3}};
for_each(nrs.begin(), nrs.end(),
    [](auto& e) {
        cout << e.first << " = "
            << e.second << '\n';
    });
```

Разрешение вызова через лямбду (пример)

```
void f(int) {}  
void f(float) {}
```

```
vector<int> v;  
for_each(v.begin(), v.end(),  
    [](auto x) { return foo(x); });
```

Immediately Invoked Function Expression (IIFE)

- вызов лямбды сразу после создания
- может использоваться для инициализации сложных объектов
- плохо заметно в коде

IIFE

(пример)

```
int x = 1;  
int y = 1;  
[&]() { ++x; ++y; } ();  
cout << x << " " << y << endl;
```


Рекомендации

- избегайте умолчаний при захвата
- избегайте висячих ссылок при захвате
- помните, что лямбды могут копироваться

ваши вопросы

Проблема параметризации

- параметризовать в STL можно
 - контейнеры (сравнение элементов)
 - алгоритмы (сортировка, поиск и т.д.)
- параметризовать вызываемым кодом
- современный механизм - лямбды

Сортировка в обратном порядке (пример)

```
#include <algorithm>
```

```
std::sort(data.begin(), data.end(),  
    std::greater<string>());
```

Сортировка пользовательская (пример)

```
auto cmp = [&data](const auto& lhs, const  
std::string& rhs) -> bool {  
    return lhs.length() < rhs.length() ||  
        (lhs.length() == rhs.length() &&  
         lhs < rhs); };  
std::sort(data.begin(), data.end(), cmp);
```

Печать контейнера

```
#include <algorithm>

std::for_each(
    data.cbegin(), data.cend(),
    [&data](const auto& val) {
        std::cout << val << ' ';
    });
```

ваши вопросы

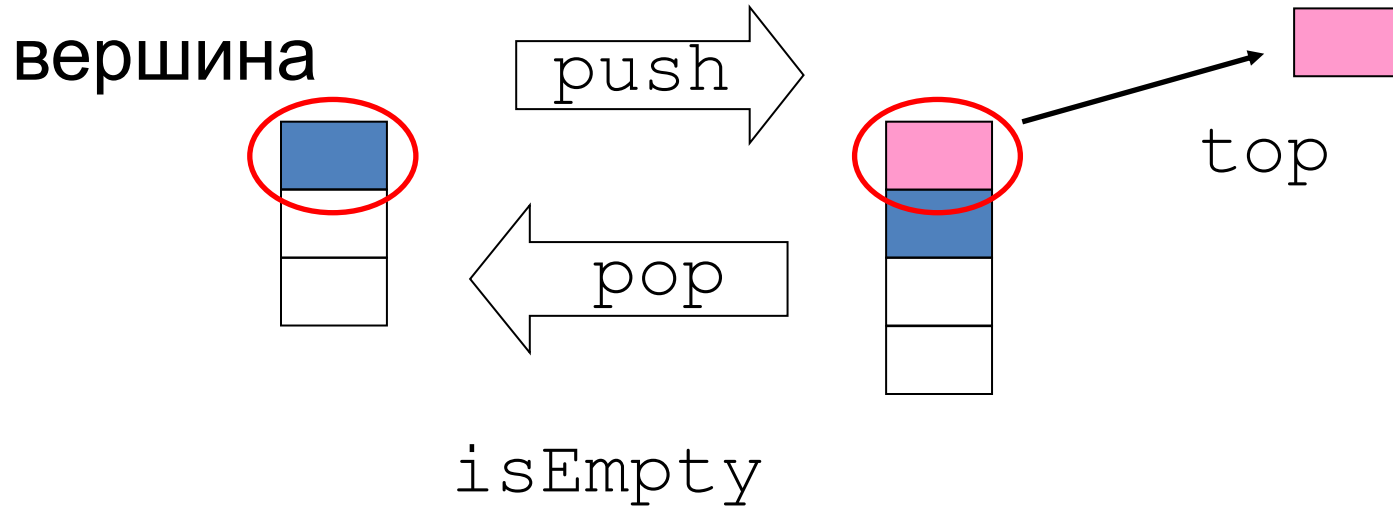
АТД «стек»

- упорядоченный набор элементов
- LIFO (Last In First Out)
- добавление и удаление элементов с одного конца (вершина)

Стек – операции

- создание/уничтожение
- добавление элемента
- удаление элемента
- проверка на пустоту
- получение значение верхнего элемента

Стек (схема работы)



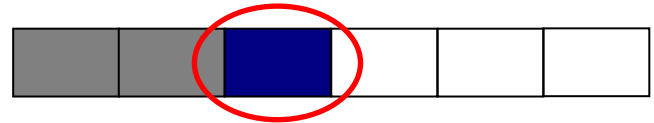
Стек (интерфейс класса)

```
Stack();  
Stack(const Stack& obj);  
~Stack();  
Stack& operator=(const Stack& rhs);  
bool isEmpty() const;  
void push(const T& v);  
void pop();  
T& top();
```

Стек

(реализация «динамический массив»)

- физический и логический размер
- добавление «в конец»
- удаление «из конца»
- проверка логического размера
- доступ к последнему элементу

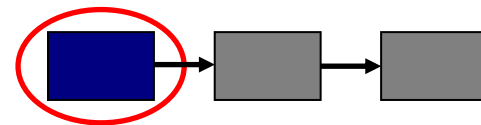


вершина

Стек

(реализация «линейный список»)

- добавление «в голову»
- удаление «из головы»
- проверка наличия элементов
- доступ к голове



«ГОЛОВА»

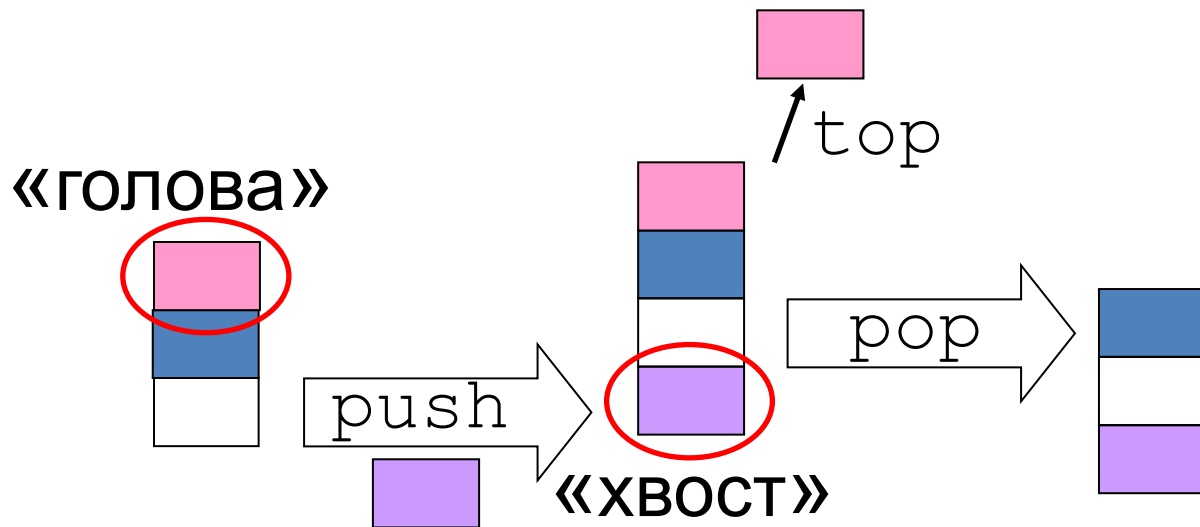
АТД «очередь»

- упорядоченный набор элементов
- FIFO (First In First Out)
 - добавление элементов с одного конца (хвост)
 - удаление с другого конца (голова)

Очередь - операции

- создание/уничтожение
- добавление элемента
- удаление элемента
- проверка на пустоту
- получение значение верхнего элемента

Очередь (схема работы)



Очередь (интерфейс класса)

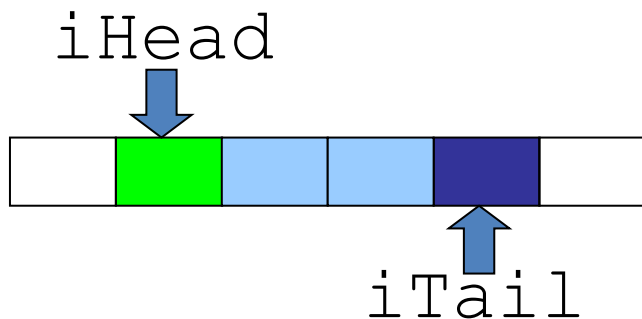
```
Queue() ;  
Queue(const Queue& obj) ;  
~Queue() ;  
Queue& operator=(const Queue& rhs) ;  
bool isEmpty() const ;  
void push(const T& v) ;  
void pop() ;  
T& top() ;
```

Очередь

(реализация «подвижные концы»)

```
iTail = (iTail + 1) % size; // push
```

```
iHead = (iHead + 1) % size; // pop
```

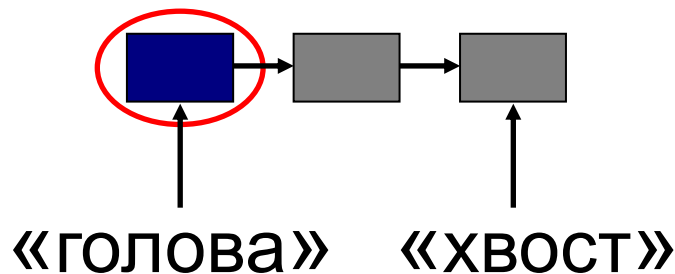


size – размер массива

Очередь

(реализация «линейный список»)

- добавление «в голову»
- удаление «из хвоста»
- проверка наличия элементов
- доступ «к хвосту»



Очередь с приоритетом

- <ключ, данные>
- порядок извлечения элементов определяется ключом, а не порядком добавления элементов в очередь

ваши вопросы

Упорядоченная пара

- кортеж длины 2

$$\langle e_1, e_2 \rangle = \{e_1, \{e_1, e_2\}\}$$

- сравнение на равенство

$$\langle x_1, x_2 \rangle = \langle y_1, y_2 \rangle \Leftrightarrow x_1 = y_1 \wedge x_2 = y_2$$

Прямое (декартово) произведение множеств

- $A \times B = \{ \langle a, b \rangle \mid \forall a \in A, \forall b \in B \}$
- прямое произведение множества A и множества B есть множество $A \times B$, элементами которого являются упорядоченные пары $\langle a, b \rangle$ для всех возможных $a \in A$ и $b \in B$

Бинарное отношение

- подмножество декартова произведения двух множеств
- R – бинарное отношение на множестве S
 $R \subset S \times S$
- записывается как
 $\langle a, b \rangle \in R$ или aRb

Отношение строгого порядка

(бинарное, антирефлексивное, транзитивное, антисимметричное отношение)

$$\forall a \neg(aRa)$$

$$\forall a \forall b \forall c \ aRb \wedge bRc \Rightarrow aRc$$

$$\forall a \forall b \ aRb \wedge bRa \Rightarrow a=b$$

обозначение ($R_<$ или $<$)

Несравнимые элементы

для множества S с заданным строгим порядком $R_<$
 $a \in S$ и $b \in S$ несравнимы, если не выполняется ни
одно из условий

$$a < b$$

$$a = b$$

$$b < a$$

Наименьший элемент

- для множества S с заданным строгим порядком $R_<$ определен наименьший элемент $a \in S$, если $\forall b \in S \ a < b$
- если определен, то единственный

Максимальный элемент

- для множества S с заданным строгим порядком $R_<$ определен максимальный элемент $a \in S$, если $\nexists b \in S \ a < b$
- может быть несколько

Дерево (через отношение)

- конечное непустое множество T узлов
- отношение $R_<$ строгого порядка на $T \times T$
- наименьший элемент (корень)
- максимальные элементы (листья)
- не максимальные элементы (внутренние узлы)

Дерево (через множества)

- конечное непустое множество T узлов
- существует один корень дерева $r \in T$
- остальные узлы (за исключением r)
распределены среди непересекающихся
множеств $T_1, \dots, T_m$, являющихся деревьями
- деревья $T_1, \dots, T_m$ называются поддеревьями
данного корня r

Дерево (через множества)

- листья – корни поддеревьев, которые не имеют подчиненных деревьев
- внутренние узлы – корни поддеревьев, которые имеют подчиненные деревья

Дерево (иллюстрация)

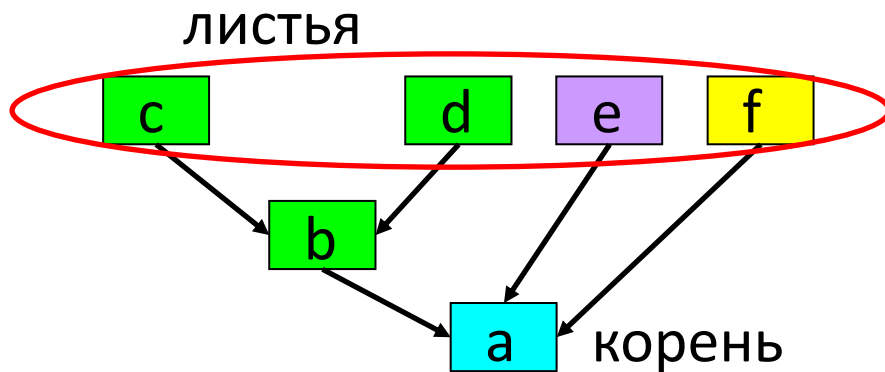
$a < b$

$a < e$

$a < f$

$b < c$

$b < d$



$\langle a, \{ \langle b, \{ \langle c \rangle, \langle d \rangle \} \rangle, \langle e \rangle, \langle f \rangle \} \rangle$

ваши вопросы

Дерево (доп. определения)

- корень – выделенная вершина дерева
- поддереву – любое дерево, корень которого не совпадает с корнем исходного дерева

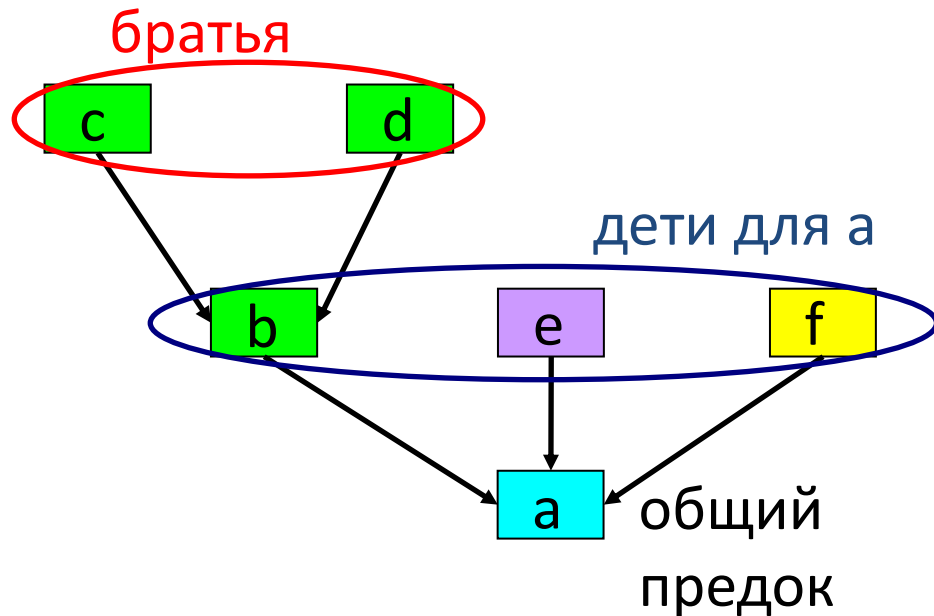
Узел дерева (доп. определения)

- степень — количество поддеревьев узла
- уровень — длина пути от корня до узла
рекурсивное определение:
 - уровень корня дерева T равен 0
 - уровень любого другого узла на единицу больше, чем уровень корня ближайшего поддерева дерева T , содержащего данный узел

«Родственные отношения»

- родитель, родительский узел
- ребенок, потомок
 - подчиненный по отношению к родителю узел
- предок
- брат
 - братья имеют общего родителя

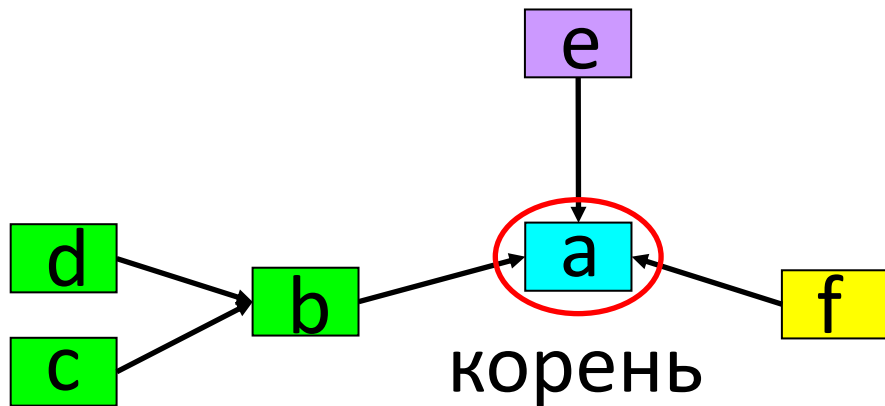
«Родственные отношения» (иллюстрация)



N-арное дерево

- степень внутреннего узла не превышает N
- степень внутреннего узла точно равна N
- говорят о
 - бинарное (двоичное) дерево, $N=2$
 - тернарное дерево, $N=3$
 - quadro-дерево, $N=4$

Радиальное подчинение (иллюстрация)

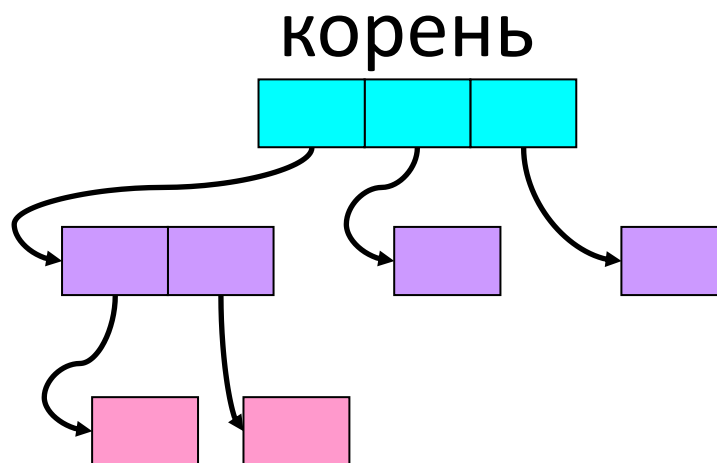


Представление дерева в памяти

- список указателей на детей
- список индексов (в массиве) детей
- «послойно»
- список ребер
- матрица отношений

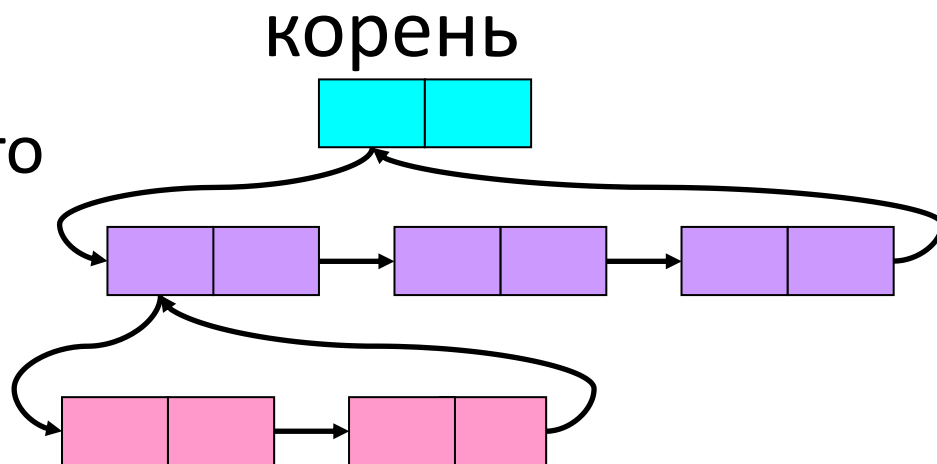
Список указателей

- узел
 - число детей
 - указатели на детей
 - данные



Послойно

- узел
 - указатель на первого ребенка
 - указатель на брата/родителя
 - данные



Список ребер

- узлы храним в массиве
- ребро – пара индексов <родитель, ребенок>
- ребра храним упорядоченно (по родителю)

Дерево (операции)

- создание и уничтожение
- добавление поддерев
- удаление поддерев
- навигация
- изменение корня

Навигация по дереву

- проверить наличие родителя / перейти к нему
- проверить наличие детей / перейти к ребенку
- перейти к брату
- перейти к корню дерева

Обход дерева

- нужен для
 - поиск заданного узла в дереве
 - получение всех узлов
- в глубину («стек для детей»)
- в ширину («очередь для детей»)

ваши вопросы